

- Description

This module manages a smart water pressure regulation system for buildings, ensuring that:

1. The system continuously monitors the current water pressure (in bar).
2. If the pressure falls below a predefined minimum threshold, the pump is activated to increase water pressure.
3. If the pressure exceeds a predefined maximum threshold, the pressure release valve is activated to reduce pressure.
4. If the pressure is within the optimal range, no action is taken.

This system helps maintain optimal water pressure in buildings, preventing damage due to overpressure and ensuring stable water supply.

- Specification

module SmartWaterPressureControl

var currentPressure, minPressure, maxPressure: real

var pumpActive, releaseValveActive: boolean

process updatePressure(newPressure: real) returns (currentPressure: real, pumpActive: boolean, releaseValveActive: boolean)

functional scenario form:

currentPressure < minPressure && pumpActive = true, releaseValveActive = false, output "ALERT: Water pressure too low (" + currentPressure + " bar). Activating water pump.", Water pump activated.

||

currentPressure > maxPressure && pumpActive = false, releaseValveActive = true, output "ALERT: Water pressure too high (" + currentPressure + " bar). Activating pressure release valve.", Pressure release valve activated.

||

currentPressure >= minPressure && pumpActive = false, releaseValveActive = false, output "Water pressure optimal (" + currentPressure + " bar). No action required.", No control action required.

end process

end module

- Java Code

```
package code;
public class c92_SmartWaterPressureSystem {
    private double minPressure;
    private double maxPressure;
    private double currentPressure;
    private boolean pumpActive;
    private boolean releaseValveActive;
    public c92_SmartWaterPressureSystem(double minPressure, double maxPressure, double initialPressure) {
```

```

        this.minPressure = minPressure;
        this.maxPressure = maxPressure;
        this.currentPressure = initialPressure;
        this.pumpActive = false;
        this.releaseValveActive = false;
    }

    public void updatePressure(double newPressure) {
        currentPressure = newPressure;
        adjustPressureControl();
        System.out.printf("Current Water Pressure: %.2f bar%n", current
Pressure);
        if (pumpActive) {
            System.out.println("ALERT: Low pressure detected. Activatin
g water pump.");
        } else if (releaseValveActive) {
            System.out.println("ALERT: High pressure detected. Activati
ng pressure release valve.");
        } else {
            System.out.println("Water pressure is optimal. No action re
quired.");
        }
    }

    private void adjustPressureControl() {
        if (currentPressure < minPressure) {
            pumpActive = true;
            releaseValveActive = false;
        } else if (currentPressure > maxPressure) {
            pumpActive = false;
            releaseValveActive = true;
        } else {
            pumpActive = false;
            releaseValveActive = false;
        }
    }

    public double getCurrentPressure() {
        return currentPressure;
    }

    public boolean isPumpActive() {
        return pumpActive;
    }

    public boolean isReleaseValveActive() {
        return releaseValveActive;
    }

    public static void main(String[] args) {

```

```

        c92_SmartWaterPressureSystem waterSystem = new c92_SmartWaterPressureSystem(2.0, 6.0, 4.0);

        // testcase-VT:
        //     waterSystem.updatePressure(1.5);
        //     waterSystem.updatePressure(4.5);
        //     waterSystem.updatePressure(6.5);

        // testcase-FT:
        //     waterSystem.updatePressure(1.7976931348623157E308);
        //     waterSystem.updatePressure(-1.7976931348623157E308);
        //     waterSystem.updatePressure(0.0);

        // testcase-Z3:
        //     waterSystem.updatePressure(1.5);
        //     waterSystem.updatePressure(4.5);
        //     waterSystem.updatePressure(6.5);

        // testcase-UVT:
        waterSystem.updatePressure(1.5);
        waterSystem.updatePressure(4.5);
        waterSystem.updatePressure(6.5);
    }
}

```

- Coverage report

1. Test case number: 3
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	834	842
✓ src	1.0 %	8	834	842
✓ code	1.0 %	8	834	842
✓ c92_SmartWaterPressureSystem.java	100.0 %	8	0	8
✓ c92_SmartWaterPressureSystem	100.0 %	8	0	8
main(String[])		0	0	0
c92_SmartWaterPressureSystem		0	0	0
adjustPressureControl()	100.0 %	4	0	4
updatePressure(double)	100.0 %	4	0	4

3. Vibration testing instruction coverage: 92%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.5 %	103	21,553	21,656
✓ src	0.5 %	103	21,553	21,656
✓ code	0.5 %	103	21,553	21,656
✓ c92_SmartWaterPressureSystem.java	92.0 %	103	9	112
✓ c92_SmartWaterPressureSystem	92.0 %	103	9	112
main(String[])	100.0 %	17	0	17
c92_SmartWaterPressureSystem	100.0 %	18	0	18
adjustPressureControl()	100.0 %	33	0	33
updatePressure(double)	100.0 %	35	0	35

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c92_SmartWaterPressureSystem.java	75.0 %	6	2	8
▼ c92_SmartWaterPressureSystem	75.0 %	6	2	8
main(String[])		0	0	0
c92_SmartWaterPressureSystem		0	0	0
adjustPressureControl()	75.0 %	3	1	4
updatePressure(double)	75.0 %	3	1	4

5. Fuzz testing instruction coverage: 83.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	94	21,562	21,656
▼ src	0.4 %	94	21,562	21,656
▼ code	0.4 %	94	21,562	21,656
▼ c92_SmartWaterPressureSystem.java	83.9 %	94	18	112
▼ c92_SmartWaterPressureSystem	83.9 %	94	18	112
main(String[])	100.0 %	17	0	17
c92_SmartWaterPressureSystem	100.0 %	18	0	18
adjustPressureControl()	81.8 %	27	6	33
updatePressure(double)	91.4 %	32	3	35

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c92_SmartWaterPressureSystem.java	100.0 %	8	0	8
▼ c92_SmartWaterPressureSystem	100.0 %	8	0	8
main(String[])		0	0	0
c92_SmartWaterPressureSystem		0	0	0
adjustPressureControl()	100.0 %	4	0	4
updatePressure(double)	100.0 %	4	0	4

7. Z3 instruction coverage: 92%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	103	22,926	23,029
▼ src	0.4 %	103	22,926	23,029
▼ code	0.4 %	103	22,926	23,029
▼ c92_SmartWaterPressureSystem.java	92.0 %	103	9	112
▼ c92_SmartWaterPressureSystem	92.0 %	103	9	112
main(String[])	100.0 %	17	0	17
c92_SmartWaterPressureSystem(double)	100.0 %	18	0	18
adjustPressureControl()	100.0 %	33	0	33
updatePressure(double)	100.0 %	35	0	35

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c92_SmartWaterPressureSystem.java	100.0 %	8	0	8
▼ c92_SmartWaterPressureSystem	100.0 %	8	0	8
main(String[])		0	0	0
c92_SmartWaterPressureSystem		0	0	0
adjustPressureControl()	100.0 %	4	0	4
updatePressure(double)	100.0 %	4	0	4

9. Unimproved V-method instruction coverage: 92%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	103	22,926	23,029
▼ src	0.4 %	103	22,926	23,029
▼ code	0.4 %	103	22,926	23,029
▼ c92_SmartWaterPressureSystem.java	92.0 %	103	9	112
▼ c92_SmartWaterPressureSystem	92.0 %	103	9	112
main(String[])	100.0 %	17	0	17
c92_SmartWaterPressureSystem(double)	100.0 %	18	0	18
adjustPressureControl()	100.0 %	33	0	33
updatePressure(double)	100.0 %	35	0	35

